DEFFI :

```
<!DOCTYPE html>
<html>
<head>
<title>Diffie Hellman Key Exchange</title>
<script>
function power(base, exp, mod) {
let result = 1;
base = base % mod;
for (let i = 0; i < exp; i++)
result = (result * base) % mod;
return result;
}
function diffieHellman() {
let p = parseInt(prompt("Enter a prime number (p):"));
let g = parseInt(prompt("Enter primitive root (g):"));
let a = parseInt(prompt("Enter private key for Alice (a):"));
let b = parseInt(prompt("Enter private key for Bob (b):"));
// Public keys
let A = power(g, a, p);
let B = power(g, b, p);
document.write("<h3>Public values:</h3>");
document.write("Prime number (p): " + p + "<br>");
document.write("Primitive root (g): " + g + "<br>");
document.write("Alice public key (A): " + A + "<br>");
document.write("Bob public key (B): " + B + "<br><br>");
// Shared secret keys
let keyA = power(B, a, p);
let keyB = power(A, b, p);
```

```
document.write("<h3>Secret keys:</h3>");
document.write("Alice's secret key: " + keyA + "<br>");
document.write("Bob's secret key: " + keyB + "<br><br>");
if (keyA === keyB)
document.write("<b>✓ Shared secret key established successfully!</b>");
else
document.write("<b>✗ Keys do not match!</b>");
}
</script>
</head>
<body>
<h2>Diffie–Hellman Key Exchange Simulation</h2>
<button onclick="diffieHellman()">Start Exchange</button>
</body>
</html>
```

RC4:

```
import java.util.*;
public class RC4 {
private byte[] S = new byte[256];
private byte[] key;
public RC4(byte[] key) {
this.key = key;
init();
}
// Key Scheduling Algorithm (KSA)
private void init() {
for (int i = 0; i < 256; i++) S[i] = (byte) i;
int j = 0;
for (int i = 0; i < 256; i++) {
```

```
j = (j + S[i] + key[i % key.length]) & 0xFF;
swap(i, j);
}
}
// Swap helper
private void swap(int i, int j) {
byte temp = S[i];
S[i] = S[j];
S[j] = temp;
}
// Pseudo-Random Generation Algorithm (PRGA)
public byte[] encryptDecrypt(byte[] data) {
int i = 0, j = 0;
byte[] result = new byte[data.length];
for (int k = 0; k < data.length; k++) {
i = (i + 1) & 0xFF;
j = (j + S[i]) & 0xFF;
swap(i, j);
int rnd = S[(S[i] + S[j]) & 0xFF];
result[k] = (byte) (data[k] ^ rnd);
}
return result;
}
// Main Method
public static void main(String[] args) {
String keyStr = "secretkey";
String plainText = "Hello World";
RC4 rc4 = new RC4(keyStr.getBytes());
byte[] cipher = rc4.encryptDecrypt(plainText.getBytes());
```

```
System.out.println("Encrypted (Hex): " + toHex(cipher));
// Decrypt
RC4 rc4Dec = new RC4(keyStr.getBytes());
byte[] decrypted = rc4Dec.encryptDecrypt(cipher);
System.out.println("Decrypted: " + new String(decrypted));
}
// Convert bytes to hex for display
private static String toHex(byte[] bytes) {
StringBuilder sb = new StringBuilder();
for (byte b : bytes)
sb.append(String.format("%02X ", b));
return sb.toString();
}
}
```

RIJANDEL:

```
import javax.crypto.*;
import javax.crypto.spec.*;
import java.util.*;
public class AESGCM {
public static void main(String[] args) throws Exception {
String msg = "Hello AES Rijndael GCM!";
// Generate key & IV
KeyGenerator kg = KeyGenerator.getInstance("AES");
kg.init(128);
SecretKey key = kg.generateKey();
byte[] iv = new byte[12];
new java.security.SecureRandom().nextBytes(iv);
// Encrypt
Cipher enc = Cipher.getInstance("AES/GCM/NoPadding");
```

```
enc.init(Cipher.ENCRYPT_MODE, key, new GCMParameterSpec(128, iv));
byte[] ct = enc.doFinal(msg.getBytes());
String cipherText = Base64.getEncoder().encodeToString(iv) + ":" +
Base64.getEncoder().encodeToString(ct);
System.out.println("Encrypted: " + cipherText);
// Decrypt
String[] parts = cipherText.split(":");
byte[] iv2 = Base64.getDecoder().decode(parts[0]);
byte[] data = Base64.getDecoder().decode(parts[1]);
Cipher dec = Cipher.getInstance("AES/GCM/NoPadding");
dec.init(Cipher.DECRYPT_MODE, key, new GCMParameterSpec(128, iv2));
System.out.println("Decrypted: " + new String(dec.doFinal(data)));
}
}
```